Teaching BrickGPT New Bricks: Expanding a Brick Library In Context vs. In Weights

Team: **[Name 1], [Name 2], [Name 3], [Name 4]** (Topic 2: In-Context Learning

Abstract

BrickGPT [1] generates stable, buildable brick structures from text, but only with eight small bricks (1×1 to 2×6). We ask whether this specialist model can learn to use *new* brick types (e.g. 4×4, 2×8) from its context alone, and how this compares with putting the new bricks into its weights. We create new-brick data automatically by merging adjacent bricks in the existing StableText2Brick dataset, so no new labels are needed. We compare zero-shot descriptions, few-shot demonstrations, LoRA, and a LoRA trained to "read the brick library" from context, tested on unseen brick types. This is a concrete, checkable case of the course question: when should information live in the context, and when in the weights?

Introduction and Background

- **BrickGPT** [1] fine-tunes Llama-3.2-1B-Instruct to predict the next brick as text on a 20^3 grid, with validity checks and physics-aware rollback; it names its "fixed set of commonly used toy bricks" as a limitation.
- **Garg et al.** [2]: transformers learn simple function classes in context and match optimal estimators. **Brown et al.** [3]: LLMs do new tasks from a few in-context examples.
- **Wei et al.** [4]: large models override semantic priors with in-context examples, but small and instruction-tuned models mostly cannot; BrickGPT is small and heavily fine-tuned.
- **Agarwal et al.** [5]: with hundreds of examples, ICL overrides pretraining biases and approaches fine-tuning.
- **Lampinen et al.** [6]: with matched data, ICL generalizes more flexibly than fine-tuning; adding in-context inferences to fine-tuning data closes the gap.
- **Chan et al.** [7]: ICL emerges from diverse, bursty data and competes with in-weights learning. **Singh et al.** [8]: ICL can fade as in-weights solutions take over.
- **Hu et al.** [9]: LoRA, our in-weights method and BrickGPT's own training method. **Akyürek et al.** [10]: training on in-context examples at test time beats plain ICL on new tasks.

Key Questions

1. **Zero/few-shot:** Given a description of new bricks (and optionally k demonstrations), does BrickGPT use them, and do structures stay valid and stable?
2. **Context vs. weights:** At matched data, how does in-context use compare with LoRA on the same examples, in usage, quality and cost?
3. **Generalization:** If trained to read a brick library from its context, can the model use brick types never seen in training? Plain LoRA cannot, by construction.

Hypothesis

Zero-shot description barely changes behavior: the fine-tuned prior for the 8 bricks dominates [4]. Few-shot raises usage but lowers validity. Plain LoRA is best on the bricks it was trained on but does not transfer. The library-in-context LoRA approaches plain LoRA on

seen bricks and clearly beats all others on held-out bricks [6].

Methods

Task design. Any shape can be built from 1×1 bricks, so large bricks must help: the prompt gives a *brick budget* ("at most N bricks", N below the 8-brick solution), and brick count and stability are metrics, since large bricks bridge seams with fewer pieces.

Data (no new labels). From StableText2Brick (47k structures, 28k ShapeNet objects, GPT-4o captions), greedily merge adjacent aligned bricks into larger types (e.g. 4×4, 2×8, 4×6). Shape and caption are unchanged; we re-check stability and keep stable structures. New types are split into *seen* (train) and *held-out* (test only). We extend the brick library in BrickGPT's validity checker, stability analysis and rendering.

Conditions (BrickGPT checkpoint): (A) original 8-brick model; (B) zero-shot description; (C) few-shot, $k \in \{1, 2, 5, 10\}$; (D) LoRA on n merged examples, n matched to C and scaled to 100–5k; (E) library-in-context LoRA with randomly sampled libraries, tested on held-out bricks.

Metrics. BrickGPT's % valid, % stable, mean/min stability, CLIP, DINO; plus new-brick usage rate, brick count, budget satisfaction, voxel IoU vs. ground truth. Train/val loss curves for D and E; error bars over 3 seeds and 100+ test prompts.

Compute. BrickGPT reports 3 epochs on 47k structures in about 12 h on 8 RTX A6000s. Our LoRA runs use 100–5k examples on a 1B model, so a few GPU-hours each, about **[X]** GPU-hours in total; inference with rollback is the main cost. Available: **[GPU type, count, hours/week — TO BE FILLED]**.

Code bases. BrickGPT repo (MIT; TRL/Accelerate fine-tuning, stability analysis, rendering), Hugging Face PEFT.

References

[1] A. Pun, K. Deng, R. Liu, D. Ramanan, C. Liu, J.-Y. Zhu. Generating Physically Stable and Buildable Brick Structures from Text. ICCV 2025 (Marr Prize).

[2] S. Garg et al. What Can Transformers Learn In-Context? A Case Study of Simple Function Classes. NeurIPS 2022.

[3] T. Brown et al. Language Models are Few-Shot Learners. NeurIPS 2020.

[4] J. Wei et al. Larger Language Models Do In-Context Learning Differently. arXiv:2303.03846, 2023.

[5] R. Agarwal et al. Many-Shot In-Context Learning. NeurIPS 2024.

[6] A. K. Lampinen et al. On the Generalization of Language Models from In-Context Learning and Finetuning. arXiv:2505.00661, 2025.

[7] S. C. Y. Chan et al. Data Distributional Properties Drive Emergent In-Context Learning in Transformers. NeurIPS 2022.

[8] A. K. Singh et al. The Transient Nature of Emergent In-Context Learning in Transformers. NeurIPS 2023.

[9] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022.

[10] E. Akyürek et al. The Surprising Effectiveness of Test-Time Training for Few-Shot Learning. arXiv, 2024.